

Qué contar en cada diapositiva. Una o dos ideas por pantalla: las capturas hablan por sí solas.

## INTRODUCCIÓN

### 01 Portada

Presento la práctica: montar el entorno de trabajo de SmartTour para que cualquiera pueda levantarlo con un solo comando.

### 02 Encargo

Leo la cita del responsable: el objetivo es un entorno reproducible. Para eso usamos cuatro herramientas: GitHub, Docker, Docker Compose y JupyterLab.

## PREPARACIÓN DEL EQUIPO

### 03 Instalar herramientas

Desde PowerShell con `winget`: Python, Node, Git y Docker Desktop, cada uno con un comando. Lo compruebo con `--version` (Python 3.10, Node 24, Git 2.51, Docker 28).

### 04 Configurar Git

`user.name` y `user.email` con `--global`: firman cada commit, y el correo debe ser el de GitHub para que los commits salgan vinculados al perfil. `git remote -v` muestra la conexión con el repositorio.

## DESARROLLO · PASOS 1-9

### 05 Repositorio y estructura

Repo público con README, licencia MIT y `.gitignore`. Explico para qué sirve cada carpeta y que Git no guarda carpetas vacías, por eso llevan `.gitkeep`.

### 06 Docker instalado

`docker --version` confirma la instalación. `hello-world` demuestra que Docker descarga una imagen y crea un contenedor.

### 07 docker-compose.yml

Recorro el archivo: la imagen, el puerto 8888 y los volúmenes, que conectan las carpetas locales con el contenedor. Aviso de que el token vacío solo vale para trabajar en local.

### 08 Contenedor arrancado

`docker compose up -d` lo levanta, `docker ps` lo muestra «healthy» y los logs confirman que Jupyter está en marcha.

## 09 JupyterLab

Se abre en `localhost:8888`. La carpeta `work/` es nuestra carpeta `notebooks/`. Destaco que no hemos instalado Python en el equipo.

---

## 10 Primer notebook

Se importan pandas, numpy y matplotlib y sale «Entorno SmartTour funcionando». Menciono las versiones: pandas 2.1.1 y numpy 1.24.4.

---

## 11 GitHub

`add`, `commit` y `push`. En la captura se ve el commit en el repositorio.

---

## 12 Checklist

Resumen rápido: todos los pasos están completados.

---

## PUNTO 7 · REFLEXIONES

## 13 ¿Por qué Docker?

El mismo entorno en todas las máquinas, la posibilidad de escalar a clústeres y adiós al «en mi máquina funciona».

---

## 14 Contenedor frente a instalación manual

Cuatro ventajas: reproducible, rápido, aislado y desechable.

---

## 15 Organizar las carpetas

Cualquiera encuentra cada cosa sin preguntar. Además, separar datos y código ayuda con Git y con Docker.

---

## 16 Imagen frente a contenedor

Explico el diagrama: la imagen es la plantilla de solo lectura y el contenedor es la instancia en marcha. Cierro con la analogía de la clase y el objeto.

---

## CIERRE

## 17 Conclusión

Nuestra oficina digital ya está lista: repositorio, contenedores y JupyterLab preparados. Con `git clone` y `docker compose up`, cualquiera del equipo puede empezar a trabajar.

---